

FrostPuno: arquitectura distribuida y cómputo paralelo para la predicción de heladas mediante aprendizaje no supervisado en el altiplano de Puno

Mamani Mendoza, Joseph Elvis Pari Pari, Yimmy Ronaldo Quispe Galindo, Jahan Kevin
Flores Curasi, Héctor Luis Apaza Llanos, Yoel
Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional del Altiplano
Puno, Perú

Resumen—Las heladas del altiplano peruano afectan cultivos y ganado, pero también constituyen el insumo del chuño, alimento deshidratado por congelamiento nocturno. Las decisiones productivas se toman por experiencia empírica, sin información climática integrada. Este trabajo presenta FrostPuno, un sistema distribuido que agrupa los regímenes térmicos de trece distritos de Puno mediante K-Means y expone el resultado como servicio en producción. El aporte se concentra en tres decisiones de cómputo paralelo y distribuido: la ingesta climática se paraleliza con un conjunto de hilos que aísla los fallos por distrito; la arquitectura se descompone en cuatro servicios autónomos alojados en proveedores distintos y acoplados solo por contratos REST; y el mantenimiento del componente inteligente se automatiza como trabajos concurrentes de integración continua con una barrera de calidad. La selección de características elevó el coeficiente de silueta de 0.286 a 0.419 y el índice de Davies-Bouldin descendió a 0.813. El sistema se encuentra desplegado y accesible como aplicación web, interfaz de programación y aplicación Android.

Index Terms—cómputo paralelo, sistemas distribuidos, aprendizaje no supervisado, K-Means, integración continua, heladas, altiplano

I. INTRODUCCIÓN

En la región Puno, por encima de los 3800 metros sobre el nivel del mar, las heladas presentan una doble condición. Dañan los cultivos y provocan mortalidad neonatal en crías de alpaca y ovino, patrimonio productivo de las familias altoandinas. Al mismo tiempo, constituyen el insumo indispensable del chuño, papa deshidratada mediante congelamiento nocturno y secado diurno que se conserva durante años y forma parte de la seguridad alimentaria regional.

La decisión de proteger un cultivo, resguardar el ganado o tender la papa se toma habitualmente por experiencia empírica. No existe una herramienta que integre información climática abierta y la traduzca en recomendaciones territoriales. El problema no se limita al modelado: exige recolectar datos de múltiples ubicaciones, servirlos con baja latencia a usuarios dispersos y mantener el sistema operativo sin intervención constante.

Este artículo describe FrostPuno desde la perspectiva del cómputo paralelo y distribuido. Las contribuciones son las siguientes:

1. Una estrategia de ingesta paralela por hilos con aislamiento de fallos por ubicación, adecuada a una carga dominada por latencia de red.
2. Una arquitectura distribuida en cuatro servicios autónomos, desplegados en proveedores distintos y acoplados únicamente por contratos explícitos.
3. Un esquema de mantenimiento automatizado del componente inteligente, ejecutado como trabajos concurrentes de integración continua y protegido por una barrera de calidad verificada mediante pruebas.

II. TRABAJOS RELACIONADOS

La predicción de heladas se ha abordado tradicionalmente con modelos supervisados que requieren registros históricos etiquetados de eventos ocurridos. En territorios donde esos registros no existen o no son accesibles de forma programática, el agrupamiento ofrece una alternativa: permite descubrir regímenes climáticos sin necesidad de etiquetas.

En cuanto a la operación de sistemas con componentes de aprendizaje, la práctica de separar el entrenamiento por lotes de la inferencia en línea, y de intermediar ambos mediante un registro de modelos, se ha consolidado como patrón de referencia. FrostPuno adopta ese patrón y lo complementa con una barrera de calidad que impide promover modelos degradados, aspecto que suele quedar fuera de los trabajos académicos de alcance similar.

III. ARQUITECTURA DISTRIBUIDA

El sistema se descompone en cuatro servicios autónomos que se comunican por HTTP siguiendo contratos REST estables. Cada uno reside en una plataforma distinta, de modo que la distribución es efectiva y no una separación lógica dentro de un mismo proceso.

El desacoplamiento se sostiene en dos decisiones. La primera es que el cliente conoce únicamente la dirección del servicio, inyectada en tiempo de compilación, por lo que el mismo código fuente produce la versión web, la versión de desarrollo y la aplicación Android. La segunda es que el servicio organiza su código en capas sucesivas de rutas,

Cuadro I
DISTRIBUCIÓN DE LOS COMPONENTES DEL SISTEMA

Componente	Plataforma	Responsabilidad
Cliente Flutter	Vercel, APK	Presentación, ubicación
Servicio FastAPI	Render	Inferencia y reglas
Base de datos	Supabase	Persistencia
Fuente climática	Open-Meteo	Datos externos

servicios y repositorios, lo que permite sustituir la persistencia sin alterar los controladores.

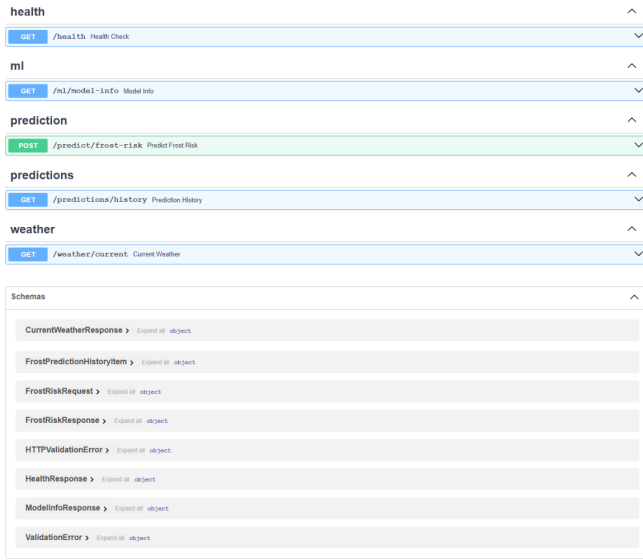


Figura 1. Contrato REST publicado por el servicio en producción.

IV. PARALELISMO EN LA INGESTA DE DATOS

La construcción del conjunto de datos requiere recolectar series climáticas horarias para trece distritos. Cada solicitud es independiente y su costo está dominado por la latencia de red, no por el procesador, de modo que la carga es adecuada para paralelismo por hilos.

El listado 1 muestra la implementación. Se lanzan tantas tareas como distritos y se recolectan conforme finalizan, sin respetar el orden de emisión.

Listing 1. Ingesta paralela con aislamiento de fallos

```
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = {
        executor.submit(fetch_weather_for_location,
                        record, start_date, end_date): record
        for record in records
    }
    for future in as_completed(futures):
        record = futures[future]
        try:
            frames.append(future.result())
        except Exception as exc:
            failures.append(
                f"{record['ubigeo']} {record['distrito']}: {exc}"
            )
```

Se destacan cuatro propiedades. La primera es el patrón de dispersión y recolección: la construcción del diccionario de tareas dispersa el trabajo y la iteración sobre las tareas completadas lo recolecta a medida que concluye. La segunda es el aislamiento de fallos: la captura de excepciones se realiza por tarea, de manera que la indisponibilidad de un distrito no interrumpe la recolección de los restantes y queda registrada de forma individual. La tercera es que el grado de paralelismo es configurable mediante un parámetro, ajustable según el ancho de banda disponible y las restricciones del proveedor de datos. La cuarta se refiere al rendimiento: siendo la carga dependiente de entrada y salida, el tiempo total tiende al de una sola solicitud en lugar del producto del número de distritos por ese tiempo, comportamiento consistente con la fracción paralelizable descrita por la ley de Amdahl.

V. DISTRIBUCIÓN DEL CÓMPUTO DE APRENDIZAJE

El sistema separa el cómputo intensivo del cómputo sensible a la latencia, siguiendo el patrón habitual en sistemas distribuidos de aprendizaje automático.

El entrenamiento se ejecuta por lotes y fuera de línea. Recorre una búsqueda exhaustiva de veinticuatro combinaciones de hiperparámetros y serializa el resultado en un registro de modelos junto con sus metadatos. La inferencia, en cambio, ocurre en línea: el servicio carga ese mismo artefacto una única vez por proceso y, ante cada solicitud, asigna el vector climático al centroide más próximo, operación de costo proporcional al número de grupos y por tanto muy económica.

La pieza que articula ambos extremos es la lista de características declarada en los metadatos. El servicio construye el vector de entrada siguiendo ese orden, por lo que un reentrenamiento no obliga a modificar el código del servidor. Este contrato constituye la frontera entre el subsistema de aprendizaje y el subsistema de servicio.

Adicionalmente, el servicio incorpora tres mecanismos de eficiencia. Emplea un servidor concurrente que atiende múltiples solicitudes simultáneas; mantiene una caché en memoria indexada por coordenada con tiempo de expiración, lo que evita consultar la fuente externa de forma repetida; e instancia una sola vez por proceso el registro de modelos, el servicio de predicción y el proveedor climático, evitando recargar el artefacto serializado en cada solicitud.

VI. MODELO DE AGRUPAMIENTO

VI-A. Conjunto de datos y características

La fuente climática es Open-Meteo, de la que se obtienen series horarias para trece distritos, resultando un conjunto de 4368 registros. La información territorial procede de una semilla compatible con el Instituto Nacional de Estadística e Informática.

Se partió de ocho variables climáticas. Un experimento de subconjuntos evidenció que la precipitación, la velocidad del viento, la nubosidad y la temperatura aparente introducían ruido o resultaban colineales, degradando la separación entre grupos. Al retirarlas, el coeficiente de silueta ascendió de 0.286 a 0.419, una mejora cercana al 47 por ciento obtenida mediante

reducción y no mediante incorporación de información. Las cuatro características conservadas son la altitud estimada, la temperatura, el punto de rocío y la humedad relativa.

VI-B. Optimización de hiperparámetros

El algoritmo es K-Means, precedido por un escalado estandarizado necesario dado que las variables presentan magnitudes heterogéneas. La configuración se determinó mediante búsqueda exhaustiva que maximiza el coeficiente de silueta.

Cuadro II
ESPACIO DE BÚSQUEDA Y CONFIGURACIÓN SELECCIONADA

Hiperparámetro	Valores	Elegido
Número de grupos	3 a 8	3
Inicialización	k-means++, aleatoria	aleatoria
Reinicios	10, 25	10
Semilla	fija	42

VI-C. Evaluación

Al tratarse de aprendizaje no supervisado no se dispone de etiquetas de referencia, por lo que no procede el uso de exactitud ni de la medida F1. La evaluación cuantifica la calidad de la estructura descubierta.

Cuadro III
SELECCIÓN DEL NÚMERO DE GRUPOS

Grupos	Silueta	Davies-Bouldin
3 (elegido)	0.4195	0.813
4	0.3894	0.906
5	0.3760	0.820
6	0.3697	0.870
7	0.3645	0.865
8	0.3391	0.883

La configuración de tres grupos resulta superior en ambas medidas y coincide con los tres niveles de riesgo que requiere la aplicación. La asignación de niveles se realiza con posterioridad al entrenamiento, ordenando los grupos por su temperatura media: el más frío corresponde al riesgo alto. Se trata de una descripción de los grupos hallados y no de una supervisión, dado que el entrenamiento no accede a esas etiquetas.

El resultado admite interpretación geográfica. El distrito de Macusani, situado a 4315 metros, queda clasificado en riesgo alto, mientras que Sandia, ubicado en valle a 2170 metros, corresponde al riesgo bajo. Esta correspondencia con la geografía regional emerge del agrupamiento y no fue codificada mediante reglas.

VII. MANTENIMIENTO E INTEGRACIÓN CONTINUA

El mantenimiento de un sistema con componentes de aprendizaje no se limita al código. Comprende el reentrenamiento, el registro de versiones, el almacenamiento de características y el control de calidad previo a la promoción de un modelo.



Figura 2. Agrupamiento de distritos con las métricas del modelo en producción.

VII-A. Trabajos concurrentes

Se implementaron cinco flujos independientes que se ejecutan de forma simultánea en entornos aislados ante cada incorporación de cambios. El primero entrena el modelo y publica los artefactos; el segundo aplica la barrera de calidad y ejecuta las pruebas de mantenimiento; el tercero verifica el servicio; el cuarto valida los conjuntos de datos; y el quinto analiza y compila el cliente. Al no existir dependencias entre ellos, el tiempo total corresponde al del flujo más lento y no a la suma de todos.

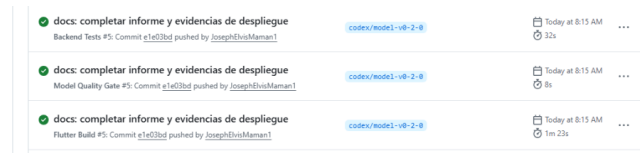


Figura 3. Ejecuciones concurrentes de los flujos de integración continua.

VII-B. Reentrenamiento adaptativo

El flujo de entrenamiento cuenta con una programación semanal, por lo que el modelo incorpora datos recientes sin intervención humana. El procedimiento no se limita a repetir el entrenamiento: si la calidad obtenida no alcanza el objetivo definido, amplía la ventana de datos históricos de catorce a veintinueve y luego a cuarenta y cuatro días, y reintenta. Cada ejecución se registra en un historial que documenta la evolución de la calidad frente al volumen de datos disponible.

VII-C. Barrera de calidad

El reentrenamiento automático introduce el riesgo de promover un modelo inferior al vigente. Para evitarlo se verifica el coeficiente de silueta antes de cualquier promoción; si el valor se sitúa por debajo de 0.35, el proceso finaliza con código de error y la integración se detiene.

La existencia de la barrera no demuestra por sí sola su funcionamiento. Se implementaron seis pruebas automatizadas que se ejecutan dentro de la integración continua y verifican el ciclo completo: la generación de artefactos, la capacidad de carga e inferencia del modelo serializado, la cobertura territorial, la documentación de la búsqueda de hiperparámetros, la aprobación del modelo vigente y, de manera particular, el rechazo de un modelo deliberadamente degradado. Esta última prueba construye un conjunto de metadatos con una medida insuficiente y comprueba que el control lo bloquea.

VIII. RESULTADOS

El sistema se encuentra desplegado y operativo. La tabla IV resume las verificaciones realizadas.

Cuadro IV
VERIFICACIONES SOBRE EL SISTEMA DESPLEGADO

Verificación	Resultado
Pruebas del servicio	12 satisfactorias
Pruebas de mantenimiento	6 satisfactorias
Barrera de calidad	Aprobada, 0.419
Compilación del cliente web	Satisfactoria
Aplicación Android	47.3 MB
Modelo en producción	K-Means v1.0.0

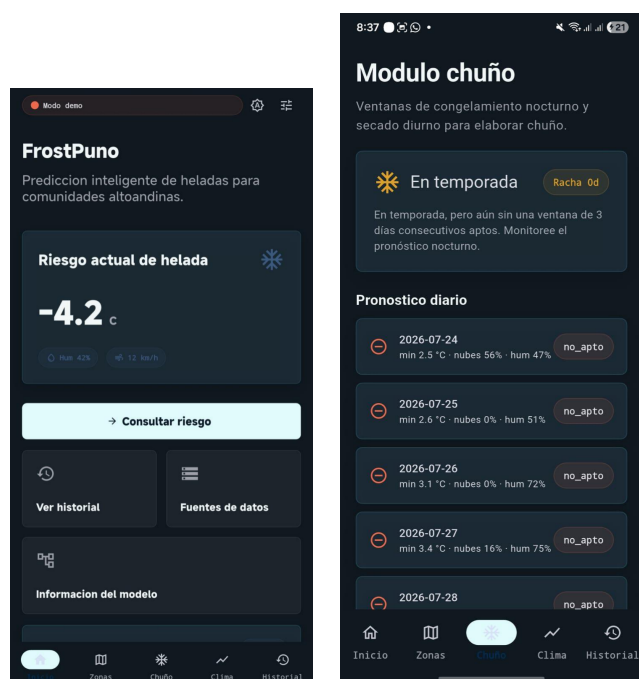


Figura 4. Cliente en operación: alerta diaria y módulo estacional de chuño.

La aplicación entrega alerta diaria de helada con umbral configurable, aviso diferenciado para ganado, detección de eventos inusuales por comparación con la climatología reciente, agrupamiento territorial, módulo estacional de chuño e histórico climático. Incorpora además selección de idioma entre español e inglés.

IX. LIMITACIONES

Los niveles de riesgo se derivan del perfil térmico de cada grupo y no de un registro oficial de heladas observadas, dado que no existe un conjunto etiquetado disponible para estos distritos. El Servicio Nacional de Meteorología e Hidrología se declara como fuente oficial prioritaria en el sistema, si bien no expone una interfaz pública estable, por lo que Open-Meteo constituye la fuente operativa. La semilla territorial corresponde a una versión reducida. Los umbrales aplicados al chuño y al ganado responden a un diseño documentado con fuentes, sin constituir cifras oficiales. La detección de eventos inusuales emplea climatología de treinta días y no una serie multianual. Finalmente, el plan gratuito de la plataforma de alojamiento suspende la instancia tras un periodo de inactividad, lo que introduce demora en la primera solicitud.

X. CONCLUSIONES

Se presentó un sistema distribuido para la predicción de heladas en el altiplano de Puno, desplegado en producción y accesible como aplicación web, interfaz de programación y aplicación Android.

Desde la perspectiva del cómputo paralelo y distribuido, el trabajo aporta tres elementos. La ingesta climática se paraleliza mediante un conjunto de hilos con aislamiento de fallos por ubicación, aprovechando que la carga depende de la latencia de red. La arquitectura se distribuye en cuatro servicios autónomos alojados en proveedores distintos, acoplados únicamente por contratos explícitos, entre los que destaca la lista de características que separa el subsistema de aprendizaje del de servicio. El mantenimiento se automatiza como trabajos concurrentes de integración continua, con reentrenamiento programado de ventana adaptativa y una barrera de calidad cuyo funcionamiento se verifica mediante pruebas.

El modelo de agrupamiento alcanza un coeficiente de silueta de 0.419 y un índice de Davies-Bouldin de 0.813, y produce una clasificación territorial coherente con la geografía regional sin haber sido programada explícitamente.

Como trabajo futuro se plantea la incorporación de observaciones oficiales como validación cruzada del agrupamiento, la ampliación de la cobertura territorial mediante la exportación completa de distritos y la evolución de la ingesta paralela hacia un esquema distribuido en varios nodos.

DISPONIBILIDAD DEL CÓDIGO

El código fuente completo se encuentra disponible de forma pública en <https://github.com/JosephElvisMaman1/frost-puno>. La aplicación web opera en <https://frost-puno.vercel.app> y el servicio en <https://frost-puno.onrender.com>, cuya documentación interactiva reside en la ruta `/docs`. La aplicación

Android se distribuye como archivo `app-release.apk` generado desde el repositorio.

APÉNDICE A

FRAGMENTOS REPRESENTATIVOS DEL CÓDIGO

Se incluyen los fragmentos centrales de los aportes descritos. El listado 2 corresponde a la búsqueda de hiperparámetros del modelo de agrupamiento.

Listing 2. Búsqueda exhaustiva de hiperparámetros

```
for k in k_range:
    for init in init_options:
        for n_init in n_init_options:
            pipeline = build_pipeline(k, init=init, n_init=
                n_init)
            labels = pipeline.fit_predict(features)
            scaled = pipeline.named_steps["scaler"].transform(
                features)
            metrics = {
                "silhouette": float(silhouette_score(scaled,
                    labels)),
                "davies_bouldin": float(davies_bouldin_score(
                    scaled, labels)),
                "inertia": float(pipeline.named_steps["kmeans"].
                    inertia_),
            }
            grid.append({"k": k, "init": init,
                "n_init": n_init, **metrics})

best = max(grid, key=lambda row: row["silhouette"])
```

El listado 3 presenta la barrera de calidad que detiene la integración ante una regresión del modelo.

Listing 3. Barrera de calidad del modelo

```
def run(metadata_path, min_silhouette):
    silhouette = load_silhouette(metadata_path)
    print(f"Cluster quality gate: silhouette={silhouette:.6f}
    }, "
        f"minimum={min_silhouette:.6f}")
    if silhouette < min_silhouette:
        print("QUALITY GATE FAILED")
        return 1
    print("QUALITY GATE PASSED")
    return 0
```

El listado 4 corresponde a la prueba que verifica el funcionamiento de dicha barrera.

Listing 4. Verificación del bloqueo de un modelo degradado

```
def test_quality_gate_blocks_degraded_model(tmp_path,
    metadata):
    degraded = dict(metadata)
    degraded["metrics"] = {"silhouette": 0.10,
        "davies_bouldin": 3.0, "inertia": 1.0}
    bad_path = tmp_path / "cluster_metadata.json"
    bad_path.write_text(json.dumps(degraded), encoding="utf-8")

    assert run_gate(bad_path, min_silhouette=0.35) == 1
```

APÉNDICE B

REPRODUCCIÓN DE LOS RESULTADOS

Los siguientes comandos permiten reproducir el entrenamiento, la barrera de calidad y las pruebas descritas.

Listing 5. Comandos de reproducción

```
python -m ml_pipeline.clustering.train_clusters
python -m ml_pipeline.registry.check_cluster_quality --min-
    silhouette 0.35
python -m pytest ml_pipeline/tests/test_maintenance_ci.py -v
python -m ml_pipeline.clustering.adaptive_retrain --target
    0.45 --max-iters 3
pytest backend_fastapi/tests -q
```